



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS orders its priority queue only by $g(n)$ — the cost already spent to reach the node. It has no idea where the goal is, so it expands outward evenly in all directions. A* orders its queue by $f(n) = g(n) + h(n)$, where $h(n)$ is a heuristic estimate of the remaining cost to the goal. This lets A* prefer nodes that are both cheap so far *and* close to the goal, so it explores far fewer nodes. In my code, the only real difference is the tuple pushed to the heap: UCS pushes (g, pos) ; A* pushes $(g + h, g, pos, path)$.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

A heuristic is admissible if it never overestimates the true cost to the goal. In my grid, the agent can only move Up, Down, Left, or Right, one cell per step. To go from $(x1, y1)$ to $(x2, y2)$, it must make at least $|x1 - x2|$ horizontal moves and $|y1 - y2|$ vertical moves — and walls can only make the real path longer. So, Manhattan distance is always $\leq$ the true cost, which makes it admissible.

If the heuristic were not admissible (overestimating), A* could push a node on the true best path far down the queue because its f -value looks too large and expand a worse route first. It would then return a path that is not optimal — it would behave like my DFS agent, finding *a* path but not the shortest one.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No — Manhattan distance would no longer be admissible. With diagonals, one move can change both x and y at once. For example, from $(0,0)$ to $(3,3)$, the true cost is 3 diagonal moves, but Manhattan gives 6. It overestimates, so A^* would lose its optimality guarantee.

The metric to switch to is Chebyshev distance: $h = \max(|x_1 - x_2|, |y_1 - y_2|)$. For $(0,0) \rightarrow (3,3)$, it gives 3, which matches the true cost. (Euclidean distance would also be admissible since a straight line is always $\leq$ any grid path, but Chebyshev is tighter for uniform-cost 8-way movement, so it explores fewer nodes.)

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Distance to the nearest pellet is weak because it ignores all the other pellets — it can lead the agent to grab a nearby one and then have to travel far back for the rest.

A stronger, still admissible heuristic is:

$h(n) = (\text{distance from the agent to the nearest pellet}) + (\text{cost of a Minimum Spanning Tree connecting all remaining pellets, using Manhattan distance as edge weights})$

The idea: to eat every pellet, the agent must at least reach the first one, and then must travel along some route that touches every remaining pellet. Any tour that visits all pellets must be at least as long as the MST that connects them, so the estimate never overestimates. A simpler (weaker but still admissible) option is the distance to the farthest pellet, since the agent must eventually reach it. Using the MST version, the agent would naturally plan a route that sweeps through clusters of pellets instead of zig-zagging between the nearest ones.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING